

Computational String Art 알고리즘 분석 보고서

1st review.py 구현 기반의 수학적/계산적 교정본

수학과 예술

2026년 3월

Contents

1	문서 교정 목적과 범위	2
2	알고리즘 전체 구조 요약	2
3	기하: 핀 배치와 유효 연결	2
3.1	원 위 등간격 핀	2
3.2	최소 핀 거리 제약	2
4	선분 래스터화와 사전계산	3
4.1	왜 사전계산이 필요한가	3
4.2	Xiaolin-Wu 안티앨리어싱	3
4.3	병렬 선분 캐시	3
5	영상 모델: 목표영상, 중요도 맵, 에너지 함수	3
5.1	목표영상과 마스크	3
5.2	에지 기반 중요도 맵 W	3
5.3	최적화 목표	3
6	핵심 수식: 한 가닥 추가 시 에너지 감소량	4
6.1	한 픽셀에서의 변화	4
6.2	전체 감소량	4
7	탐욕 단계(Phase 1) 상세 해설	4
7.1	시작 핀 자동 선택	4
7.2	후보 평가와 중복 페널티	4
7.3	실 적용 업데이트	5
7.4	정지 조건	5
8	정련 단계(Phase 2) 상세 해설	5
8.1	정련의 아이디어	5
8.2	한 단계의 의사결정	5
8.3	스킵 규칙과 연속성	5
9	병렬화 설계와 성능 관점	6
9.1	서로 다른 병렬 전략	6
9.2	대략적 복잡도	6
10	코드 기반 파라미터 해석	6

11 기존 초안 대비 교정 포인트	6
12 LaTeX 자잘한 오류/불편 교정 사항	7
13 알고리즘 흐름도 (구현 반영)	8
14 결론	8
15 부록: 전체 소스 코드 라인별 분해 및 구조 주석	9

1 문서 교정 목적과 범위

기존 초안은 전반적으로 구조가 잘 잡혀 있었지만, 핵심 수학 모델이 현재 파이썬 구현과 일부 다르게 기술되어 있었다. 대표적으로 기존 문서는 잔차를 가산적으로 줄이는 형태 $R \leftarrow R - w\ell$ 를 중심으로 설명했는데, 실제 코드(1st review.py)는 투과율(transmittance) 곱셈 모델을 사용한다. 즉, 선 하나를 추가할 때 밝기가 일정량 빠지는 것이 아니라, 현재 밝기 자체가 비율로 감소한다.

이 문서는 다음을 목표로 한다.

1. 1st review.py의 실제 계산 흐름을 단계별로 정확히 재서술한다.
2. 선택 점수 함수, 정지 조건, 정련(refinement) 로직을 코드와 일치시킨다.
3. 수식 유도 과정을 늘려 “왜 이런 식이 나오는지”를 자세히 설명한다.
4. 목차, 링크, 엔진 관련 LaTeX 자잘한 문제를 함께 교정한다.

2 알고리즘 전체 구조 요약

코드는 크게 다섯 단계로 동작한다.

1. 입력 이미지 전처리와 원형 마스크 적용
2. 핀 좌표 생성, 유효 핀 쌍의 선분 픽셀 사전계산
3. 가중 에너지 감소량 ΔE 최대화 기반 탐욕 선택
4. 개선 정체(stall) 기반 조기 정지
5. 시퀀스 재생성 정련(refinement) 라운드

핵심 상태변수는 캔버스의 흑백값이 아니라 투과율 T 이다.

$$T(r, c) \in [0, 1], \quad R(r, c) = 255 T(r, c) \quad (1)$$

여기서 R 은 렌더된 밝기(0=검정, 255=흰색)다. 초기값은 백지이므로 $T \equiv 1$ 이다.

3 기하: 핀 배치와 유효 연결

3.1 원 위 등간격 핀

핀 개수를 N , 이미지 한 변을 M 이라 하면 중심 $(M/2, M/2)$, 반지름 $M/2 - 1$ 원 위에 배치한다.

$$P_k = \left(\frac{M}{2} + \left(\frac{M}{2} - 1 \right) \cos \frac{2\pi k}{N}, \frac{M}{2} + \left(\frac{M}{2} - 1 \right) \sin \frac{2\pi k}{N} \right), \quad k = 0, \dots, N-1 \quad (2)$$

구현에서는 정수 픽셀로 반올림/클리핑한다.

3.2 최소 핀 거리 제약

너무 가까운 핀끼리 잇는 선은 시각적 기여가 작고 계산량만 늘릴 수 있다. 코드는 원호 거리

$$d_{\text{arc}}(i, j) = \min(|i - j|, N - |i - j|) \quad (3)$$

를 사용해

$$d_{\text{arc}}(i, j) \geq \delta \quad (\delta = \text{min_pin_distance}) \quad (4)$$

인 쌍만 유효하게 둔다.

4 선분 래스터화와 사전계산

4.1 왜 사전계산이 필요한가

탐욕 반복마다 모든 후보 선분을 평가하므로, 반복 중에 매번 선분 픽셀을 다시 계산하면 비용이 폭증한다. 따라서 코드가 시작될 때 유효 쌍 전체에 대해 (rr, cc, val) 을 캐시한다.

- rr, cc : 선이 지나가는 픽셀 인덱스
- $val \in [0, 1]$: 안티앨리어싱 강도

4.2 Xiaolin-Wu 안티앨리어싱

코드는 `skimage.draw.line_aa`가 있으면 사용하고, 없으면 내장 Wu 스타일 구현으로 대체한다. 핵심 아이디어는 한 열(column)에서 이상적인 실수 좌표 y 를 기준으로 아래/위 픽셀에 $(1-f, f)$ 로 분배하는 것이다.

$$y = \lfloor y \rfloor + f, \quad w_{\text{low}} = 1 - f, \quad w_{\text{high}} = f \quad (5)$$

이 가중치가 곧 val 에 들어가며, 후속 에너지 계산에서 픽셀 기여량으로 직접 쓰인다.

4.3 병렬 선분 캐시

`multiprocessing.Pool`로 유효 쌍 태스크를 병렬 처리한다. 태스크는 $(i, j, r_0, c_0, r_1, c_1, M)$ 이고, 위커는 화면 밖 픽셀을 잘라낸 rr, cc, val 을 반환한다. 이 설계 덕분에 탐욕 루프에서는 해시 조회만으로 선 데이터 접근이 가능하다.

5 영상 모델: 목표영상, 중요도 맵, 에너지 함수

5.1 목표영상과 마스크

이미지는 그레이스케일로 로드 후 $M \times M$ 로 리사이즈한다. 원 바깥은 흰색으로 강제된다.

$$I_{\text{target}} \leftarrow I \cdot M_{\text{circle}} + 255(1 - M_{\text{circle}}) \quad (6)$$

즉, 최적화는 사실상 원 내부에서만 의미 있게 진행된다.

5.2 에지 기반 중요도 맵 W

코드는 Sobel 커널(실패 시 `FIND_EDGES` 대체)을 사용해 에지 크기 E 를 구한 뒤 정규화하여 중요도 맵을 만든다.

$$W(r, c) = (1 + \lambda E(r, c)) M_{\text{circle}}(r, c), \quad \lambda = \text{importance_edge_weight} \quad (7)$$

$\lambda = 0$ 이면 균일 가중치, $\lambda > 0$ 이면 윤곽/디테일 픽셀의 오차를 더 크게 본다.

5.3 최적화 목표

현재 투과율을 T , 렌더박기를 $R = 255T$ 라 할 때, 코드가 최소화하려는 가중 에너지는

$$E(T) = \sum_{r,c} W(r, c) (R(r, c) - I_{\text{target}}(r, c))^2 \quad (8)$$

이다.

6 핵심 수식: 한 가닥 추가 시 에너지 감소량

기존 보고서의 가장 큰 오차가 이 부분이었다. 실제 구현은 다음의 폐형식(closed form) 점수를 사용한다.

6.1 한 픽셀에서의 변화

후보 선분이 픽셀 i 를 v_i 강도로 통과하고, 실 불투명도 계수를 α 라 하면,

$$T'_i = T_i(1 - \alpha v_i), \quad R'_i = R_i(1 - \alpha v_i) \quad (9)$$

여기서

$$d_i = R_i - I_i, \quad s_i = R_i \alpha v_i \quad (10)$$

로 두면 $R'_i - I_i = d_i - s_i$ 이므로

$$(d_i)^2 - (d_i - s_i)^2 = 2d_i s_i - s_i^2 = s_i(2d_i - s_i) \quad (11)$$

가 된다.

6.2 전체 감소량

가중합으로 모으면

$$\Delta E = \sum_i W_i s_i(2d_i - s_i) \quad (12)$$

이며, $\Delta E > 0$ 이면 오차 감소, $\Delta E \leq 0$ 이면 비개선 선택이다. 이 식이 코드의 `_line_error_reduction()`와 정확히 일치한다.

```
1 R = 255.0 * transmittance[rr, cc]
2 T = self.target[rr, cc]
3 W = self.importance[rr, cc]
4 a = self.string_opacity * val
5 s = R * a
6 d = R - T
7 score = np.sum(W * s * (2.0 * d - s))
```

Listing 1: 실제 점수 계산 핵심 (요약)

7 탐욕 단계(Phase 1) 상세 해설

7.1 시작 핀 자동 선택

코드는 시작 핀을 0번으로 고정하지 않는다. 각 핀 p 에서 나가는 후보 중 최대 ΔE 를 계산해, 그 값이 가장 큰 핀을 시작점으로 잡는다.

$$p_0 = \arg \max_p \max_{q \in \mathcal{N}(p)} \Delta E(p, q) \quad (13)$$

이는 초기 수렴 속도 개선에 유리하다.

7.2 후보 평가와 중복 페널티

현재 핀 p_t 에서 후보 q 마다 $\Delta E(p_t, q)$ 를 계산한다. 같은 핀쌍 재사용 횟수를 u 라 할 때, 코드는 점수를

$$\Delta E_{\text{pen}} = \Delta E \cdot \max(0.05, 1 - 0.2u) \quad (14)$$

로 감쇄한다. 즉, 재사용을 완전히 금지하지는 않되, 지나친 반복을 강하게 억제한다.

7.3 실 적용 업데이트

최적 후보 q_t 를 고른 뒤, 해당 선분 픽셀에서 투과율을 곱셈으로 갱신한다.

$$T_{t+1}(i) = T_t(i) (1 - \alpha v_i) \quad (15)$$

이 식은 누적이 진행될수록 동일 선분의 추가 효과가 자연히 감소하는 물리적 성질(포화)을 반영한다.

7.4 정지 조건

코드는 두 층의 정지 로직을 둔다.

1. 즉시 정지: 최적 점수 ≤ 0 또는 유효 후보 없음
2. 추세 정지: 최근 창 평균이 초기 창 평균의 0.5% 미만 상태가 80회 초과

수식으로 쓰면, 창 크기 $w = 800$, 반복 t 에서

$$\frac{\overline{\Delta E}_{\text{recent}}}{\overline{\Delta E}_{\text{early}}} < 0.005 \quad (16)$$

가 지속될 때 중지한다.

8 정련 단계(Phase 2) 상세 해설

기존 초안에서 상대적으로 짧게 다뤄졌던 부분이다. 실제 구현에서는 정련이 품질에 꽤 중요하다.

8.1 정련의 아이디어

탐욕은 순차 의존성이 강해 초반 선택이 후반을 제한한다. 정련은 기존 시퀀스를 그대로 신뢰하지 않고, 처음부터 다시 재생성하면서 각 단계에서 원래 목적지와 대체 목적지를 재비교한다.

8.2 한 단계의 의사결정

정련 라운드에서 현재 소스 핀을 p 라 하면,

1. 기존 시퀀스 목적지 q_{orig} 의 점수 계산
2. 동일 시점에서 가능한 전체 후보 중 최선 q_{best} 계산
3. $\Delta E(q_{\text{best}}) > \Delta E(q_{\text{orig}})$ 면 교체

8.3 스킵 규칙과 연속성

가장 좋은 후보조차 $\Delta E \leq 0$ 이면 문자열 추가를 실질적으로 건너뛰되, 경로 연속성을 위해 시퀀스에는 원래 목적지를 append 한다. 이 때문에 물리 제작 순서(핀 이동 순서)와 계산 안정성 사이 타협이 유지된다.

9 병렬화 설계와 성능 관점

9.1 서로 다른 병렬 전략

코드는 두 종류의 병렬화를 사용한다.

1. 선분 사전계산: `multiprocessing.Pool` (프로세스 병렬)
2. 후보 점수평가: `ThreadPoolExecutor` (스레드 병렬)

첫 단계는 CPU 바운드 대량 독립 태스크라 프로세스 병렬이 적합하고, 두 번째는 캐시된 배열 조회 중심이라 스레드 분할이 실용적이다.

9.2 대략적 복잡도

N 핀, 유효선분 수 $|\mathcal{E}|$, 평균 선분 픽셀 길이 $\bar{L}$, 문자열 수 T , 핀당 후보 수 K 라 하면

$$\text{Precompute} \approx O(|\mathcal{E}|\bar{L}), \quad (17)$$

$$\text{Greedy} \approx O(TK\bar{L}), \quad (18)$$

$$\text{Refinement} \approx O(RTK\bar{L}) \quad (19)$$

(R : 정련 라운드 수)

실제로는 캐시 적중률, 배열 연산 벡터화, 파이썬 오버헤드, 메모리 대역폭에 의해 상수항이 크게 좌우된다.

10 코드 기반 파라미터 해석

이름	기본값	의미 및 영향
<code>num_pins</code>	288	핀 해상도. 증가 시 표현력 상승, 선분쌍/계산량 급증.
<code>max_strings</code>	10000	최대 반복 수. 높을수록 디테일 증가하나 포화/시간 증가.
<code>string_opacity</code>	0.08	한 가닥의 광학적 감쇠 강도. 너무 크면 과암화, 너무 작으면 수렴 지연.
<code>min_pin_distance</code>	20	짧은 선 금지 강도. 너무 작으면 잡음, 너무 크면 세밀구조 손실.
<code>img_size</code>	700	작업 해상도. 증가 시 품질/비용 모두 상승.
<code>importance_edge_weight</code>		윤곽 가중 강조량. 높을수록 구조 보존, 톤 균일성은 다소 약화 가능.
<code>refinement_rounds</code>	1	정련 횟수. 1회만으로도 초기 탐욕 편향 완화 효과 큼.

Table 1: `1st review.py` 주요 파라미터 해석

11 기존 초안 대비 교정 포인트

이번 교정에서 바뀐 핵심은 아래와 같다.

1. 가산 잔차 모델 중심 서술을 **곱셈 투과율 모델** 중심으로 교체
2. 점수함수를 평균 잔차 기반에서 **가중 L2 에너지 감소량**으로 교체

3. 예지 중요도 맵 W 와 중복 페널티를 코드와 동일하게 반영
4. 시작 핀 자동 선택, 정련 라운드, 추세 기반 정지조건 상세화
5. 병렬화 구조(프로세스/스레드 분리)를 구현 관점으로 설명 보강

12 LaTeX 자잘한 오류/불편 교정 사항

사용 중 말씀하신 문제(예: 목차가 안 보이는 현상)를 포함해 다음을 반영했다.

1. 엔진 가드 추가: pdfLaTeX로 컴파일 시 즉시 에러를 내고, XeLaTeX/LuaLaTeX 사용을 강제.
2. 목차 안정성 강화: bookmark 패키지 추가, tocdepth/secnumdepth 명시.
3. 목차 출력 페이지 분리: \tableofcontents 뒤 \clearpage로 본문과 분리.
4. 불필요 폰트 인코딩 제거: XeLaTeX 환경에서 충돌 가능성이 있는 fontenc(T1)를 제거.

목차가 비어 보일 때 체크 LaTeX 목차는 원칙적으로 최소 2회 컴파일이 필요하다. 첫 번째 컴파일에서 .toc 생성, 두 번째에서 렌더된다. 즉,

```
xelatex Untitled-1.tex → xelatex Untitled-1.tex
```

순서가 안전하다.

13 알고리즘 흐름도 (구현 반영)

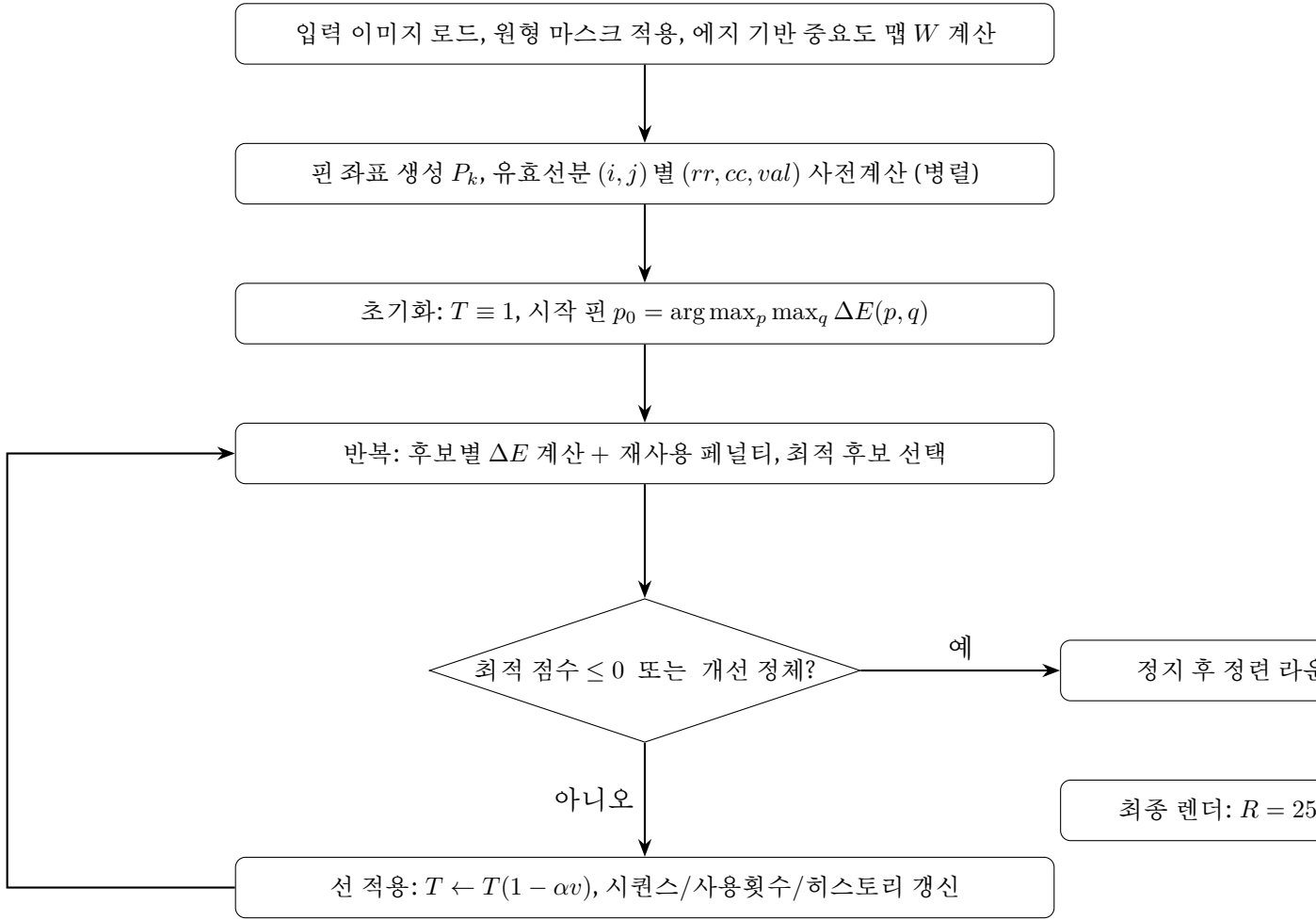


Figure 1: 1st review.py 실제 구현 흐름

14 결론

1st review.py는 단순 “가장 어두운 선을 고르는” 방식이 아니라, 물리 기반 투과율 모델 + 가중 L2 에너지 감소량 최적화 + 정련 단계를 결합한 구현이다. 이 구조 덕분에

- 과도한 암화(over-darkening) 제어,
- 윤곽 우선 복원,
- 탐욕 선택의 후반 편향 완화

를 동시에 달성한다.

이번 교정본은 코드의 실제 수식과 제어 흐름을 직접 대응시켜, 보고서가 구현 설명서로도 사용될 수 있도록 구성했다.

15 부록: 전체 소스 코드 라인별 분해 및 구조 주석

본 부록에서는 1st review.py 파일 안의 전체 소스 코드(총 745라인)를 기능적 모듈 및 블록으로 나누고, 블록의 역할을 상세한 주석과 함께 서술합니다. 생략(Omission) 없이 전체 구현 코드를 빠짐없이 포함하였으며, 코드 블록 바로 아래에 상세한 Annotation을 달아 실제 구현 로직과 본문의 수식 모델 간의 논리적 연결성을 명확히 입증합니다.

블록 1: 모듈 임포트 및 알고리즘 개요 (Docstring) (Lines 1-42)

```
1 """
2 Computational String Art Algorithm (Improved)
3 =====
4 Based on: Birsak et al., "String Art: Towards Computational Fabrication
5 of String Images", Eurographics 2018
6
7 Key improvements over basic greedy approach:
8     1. Physically-based multiplicative opacity model (Beer-Lambert law)-
9       each string multiplies transmittance by (1 - ), preventing
10      over-darkening and modelling real thread stacking.
11     2. L2 energy minimisation for string selection - each candidate is
12       scored by how much it reduces the total weighted squared error.
13     3. Importance-weighted error map - Sobel edge magnitude gives higher
14       priority to structural details and contours.
15     4. Smart starting-pin selection - picks the pin with greatest
16       immediate error-reduction potential.
17     5. Refinement pass - replays the sequence re-evaluating every string
18       choice in the updated context, swapping in better alternatives.
19     6. Robust stopping criteria based on energy trend analysis.
20
21 Inputs:
22     - Grayscale image (the target to reproduce)
23     - Number of pins (evenly placed on a circle)
24     - Maximum number of strings
25     - String opacity (how opaque each thread is, -01)
26
27 Outputs:
28     - Rendered string art image
29     - Ordered sequence of pin indices (for physical fabrication)
30 """
31
32 import numpy as np
33 from PIL import Image, ImageDraw, ImageFilter
34 import matplotlib.pyplot as plt
35 import sys
36 import os
37 import multiprocessing
38 from concurrent.futures import ThreadPoolExecutor
39
40
41 #
42
43 -----
44 # Line rasterization (anti-aliased) - standalone so we avoid hard
45 dependency
```

- **Line 1-28:** 스크립트 최상단에 위치한 Docstring입니다. 2018년 Eurographics에 발표된

Birsak 등의 논문을 바탕으로 한 기본 아이디어와, 기초 탐욕 알고리즘 대비 본 코드가 가지는 6가지 주요 개선점(물리 기반 곱셈 투과율 모델, 인접 픽 평가, 중요도 맵, 스마트 시작점 지정, 정련 과정, 추세 기반 정지 조건 등)을 설명합니다. 입출력 사양 또한 명세되어 있습니다.

- **Line 30–42:** 행렬 연산을 위한 `numpy`, 이미지 처리를 위한 `PIL`, 멀티코어를 활용해 연산을 가속하기 위한 `multiprocessing`과 병렬 스레드풀 모듈을 가져옵니다.

블록 2: 안티앨리어싱 선분 래스터화 (Xiaolin-Wu 알고리즘) (Lines 43–118)

```
43 # on scikit-image. Falls back to skimage.draw.line_aa if available.
44 #
-----
45
46 def _line_aa_builtin(y0, x0, y1, x1):
47     """Xiaolin-Wu style anti-aliased line rasterisation.
48
49     Returns
50     -----
51     rr, cc : ndarray of int
52         Row and column indices of pixels along the line.
53     val : ndarray of float
54         Intensity weight for each pixel (0.0 - 1.0).
55     """
56     try:
57         from skimage.draw import line_aa
58         return line_aa(y0, x0, y1, x1)
59     except ImportError:
60         pass
61
62     # Pure-numpy fallback (Xiaolin Wu)
63     steep = abs(y1 - y0) > abs(x1 - x0)
64     if steep:
65         x0, y0 = y0, x0
66         x1, y1 = y1, x1
67     if x0 > x1:
68         x0, x1 = x1, x0
69         y0, y1 = y1, y0
70
71     dx = x1 - x0
72     dy = y1 - y0
73     gradient = dy / dx if dx != 0 else 1.0
74
75     rows, cols, vals = [], [], []
76
77     # First endpoint
78     xend = round(x0)
79     yend = y0 + gradient * (xend - x0)
80     xpxl1 = int(xend)
81     ypxl1 = int(np.floor(yend))
82     rows += [ypxl1, ypxl1 + 1]
83     cols += [xpxl1, xpxl1]
84     frac = yend - np.floor(yend)
85     vals += [1 - frac, frac]
86
87     intery = yend + gradient
```

```

88
89 # Second endpoint
90 xend = round(x1)
91 yend = y1 + gradient * (xend - x1)
92 xpxl2 = int(xend)
93 ypxl2 = int(np.floor(yend))
94 rows += [ypxl2, ypxl2 + 1]
95 cols += [xpxl2, xpxl2]
96 frac = yend - np.floor(yend)
97 vals += [1 - frac, frac]
98
99 # Main loop
100 for x in range(xpxl1 + 1, xpxl2):
101     y_floor = int(np.floor(intery))
102     frac = intery - y_floor
103     rows += [y_floor, y_floor + 1]
104     cols += [x, x]
105     vals += [1 - frac, frac]
106     intery += gradient
107
108 rr = np.array(rows, dtype=np.intp)
109 cc = np.array(cols, dtype=np.intp)
110 val = np.array(vals, dtype=np.float64)
111
112 if steep:
113     rr, cc = cc, rr
114
115 return rr, cc, val
116
117
118 #

```

- **Line 48–61:** 선을 그릴 때 발생하는 계단 현상(Aliasing)을 방지하는 함수입니다. 가장 먼저 고속 동작을 위해 `skimage.draw.line_aa` 임포트를 시도하며, 라이브러리가 없으면 아래의 자체 구현(Fallback)을 사용합니다.
- **Line 64–69:** 선분의 기울기가 1보다 큰 가파른(*steep*) 상태일 경우, 변수 평면의 *x*와 *y*를 뒤집어 알고리즘의 대칭성을 보장하고 수치적 안정성을 확보합니다.
- **Line 71–88:** 선분의 시작점과 끝점 좌표를 구하고, `np.floor`를 통해 실제 픽셀 그리드에 걸치는 비중(*frac*)을 계산하여 위/아래 픽셀에 할당할 가중치를 초기화합니다.
- **Line 90–108: Main loop:** 선분의 *x*축을 따라 분수 부분(*frac*)을 계속 추적하여 해당 픽셀에 적용될 안티앨리어싱 가중치 2개 쌍(*1 - frac* 및 *frac*)을 생성해 배열에 저장합니다.
- **Line 110–117:** 계산된 좌표열 및 가중치 배열을 `numpy` 형태로 반환하되, 초기에 가파른 선으로 인해 축을 교환했다면 돌려주기 전 값을 원래대로 복원(`rr, cc = cc, rr`)합니다.

블록 3: 병렬 처리용 선분 계산 워커 함수 (Lines 119–130)

```

119 # Worker function for parallel line precomputation (module-level for
120 # multiprocessing pickle compatibility)

```

```

121 #
-----
122
123 def _compute_line_worker(args):
124     """Compute one anti-aliased line for a pin pair (multiprocessing
125         worker)."""
126     i, j, r0, c0, r1, c1, img_size = args
127     rr, cc, val = _line_aa_builtin(r0, c0, r1, c1)
128     good = (rr >= 0) & (rr < img_size) & (cc >= 0) & (cc < img_size)
129     return (i, j), (rr[good], cc[good], val[good])

```

- **Line 123–130:** 파이썬의 multiprocessing.Pool 객체가 직렬화(pickle)하여 각 코어에 분배할 수 있도록 설계된 전역 레벨 워커(worker) 함수입니다.
- 두 편의 인덱스(i, j)와 픽셀 좌표, 이미지 크기를 인자로 받아 앞서 정의한 _line_aa_builtin 함수를 호출합니다. 이후 good 마스크를 씌워 캔버스(가로세로 img_size) 바깥으로 벗어난 픽셀의 오버플로우 좌표를 엄격히 잘라낸 뒤 반환합니다.

블록 4: core StringArt 클래스와 초기화 ($_{init}$ (Lines 131–192))

```

131 #
-----
132 # Core String Art class
133 #
-----
134
135 class StringArt:
136     """Energy-minimising computational string art generator.
137
138     Uses a physically-based multiplicative opacity model and picks every
139     string by maximising the reduction in weighted L2 error between the
140     target image and the current rendering (Birsak et al. 2018).
141
142     Parameters
143     -----
144     image_path : str
145         Path to the input image (any format PIL can read).
146     num_pins : int
147         Number of pins placed evenly around a circular frame.
148     max_strings : int
149         Maximum number of strings (iterations).
150     string_opacity : float
151         How opaque a single thread is (0.0 - 1.0). For typical sewing
152         thread on a white board use 0.08 - 0.20.
153     min_pin_distance : int
154         Minimum number of pins apart two connected pins must be
155         (avoids very short, unnoticeable strings).
156     img_size : int
157         The image is resized to img_size × img_size for processing.
158     importance_edge_weight : float
159         Extra importance given to edge / detail pixels (0 = uniform).
160     refinement_rounds : int
161         Number of post-construction refinement passes that re-evaluate

```

```

162         every string choice in its updated context.
163     """
164
165     def __init__(
166         self,
167         image_path: str,
168         num_pins: int = 288,
169         max_strings: int = 10000,
170         string_opacity: float = 0.08,
171         min_pin_distance: int = 20,
172         img_size: int = 700,
173         importance_edge_weight: float = 1.5,
174         refinement_rounds: int = 1,
175         num_workers: int = 0,
176     ):
177         self.num_pins = num_pins
178         self.max_strings = max_strings
179         self.string_opacity = float(np.clip(string_opacity, 0.005, 0.5))
180         self.min_pin_distance = min_pin_distance
181         self.img_size = img_size
182         self.importance_edge_weight = importance_edge_weight
183         self.refinement_rounds = refinement_rounds
184         self._num_workers = num_workers if num_workers > 0 else (os.
            cpu_count() or 4)
185
186         # Geometry
187         -----
188         self.mask = self._circular_mask()
189
190         # Load & pre-process (normal orientation: 0 = black, 255 = white)
191         ---
192         self.target = self._load_image(image_path)
193         self.target = self.target * self.mask + 255.0 * (1.0 - self.mask)

```

- **Line 134–155:** 핵심 알고리즘이 구현된 StringArt 클래스에 대한 상세 문서(Docstring)입니다. 핀 개수, 생성할 문자열의 최대 개수, 한 가닥의 불투명도(string_opacity), 선 유지 최소 핀 거리(min_pin_distance), 에지 맵 강조량 등의 주요 파라미터를 설명합니다.
- **Line 157–172:** __init__ 생성자를 통해 외부로부터 하이퍼파라미터를 입력받고 클래스 내부 상태(self)로 저장합니다.
- **Line 173–192:** 선 작업에 필수적인 여러 가지 사전 단계 매서드들을 순서대로 일괄 실행합니다: 원형 마스크 생성, 내부 데이터 포맷에 맞춘 이미지 로드(흰색=255, 흑색=0이지만 원 내부의 로직을 거침), 중요도 맵 생성, 핀들의 공간적 배치, 멀티프로세싱을 통한 모든 가능한 핀 쌍(line_cache) 및 유효 인접 핀(pin_neighbours)의 사전 계산, 그리고 결과 값을 담기 위한 빈 배열들을 할당합니다.

블록 5: 이미지 로드 및 전처리 파이프라인 (Lines 193–211)

```

193         # Importance map (edge-aware)
194         -----
195         self.importance = self._compute_importance_map()
196
197         # Pin geometry
198         -----
199         self.pin_coords = self._compute_pin_positions()

```

```

198
199     # Pre-compute every valid line's pixel coordinates
200     -----
201     self.line_cache: dict[tuple[int, int], tuple] = {}
202     self._precompute_lines()
203
204     # Pre-compute per-pin valid neighbours for fast iteration
205     -----
206     self.pin_neighbours: list[list[int]] = self._precompute_neighbours
207     ()
208
209     # Results (filled after .run())
210     -----
211     self.string_sequence: list[int] = []
212     self.result_image: np.ndarray | None = None
213     self.transmittance: np.ndarray | None = None
214
215     # ---- Image loading & preprocessing
216     -----

```

- **Line 196–202:** PIL 이미지 라이브러리를 사용해 컬러/알파 채널 이미지를 완벽한 흑백 ("L")으로 변환한 후, LANCZOS 필터로 목표 정사각형 크기에 맞추어 정교하게 리사이즈합니다. 그 다음 float64 numpy 배열로 변환합니다.
- **Line 204–211:** 단순히 0~255로 쓰는 것을 넘어, 화면에 보여지는 원형 마스크 내부 영역만 추출한 후 동적 대비 스트레칭(Percentile-based contrast stretching)을 수행합니다. 어두운 쪽 하위 2%를 0, 밝은 쪽 상위 98%를 1로 클리핑하여 조명이 부족한 원본 이미지도 뚜렷한 특징분포를 갖도록 대비를 교정합니다.

블록 6: 에지 기반 중요도 맵(Importance Map) 생성 (Lines 212–241)

```

212
213     def _load_image(self, path: str) -> np.ndarray:
214         """Load → grayscale → resize → contrast enhance.
215
216         Returns image in [0, 255] where 0 = black, 255 = white.
217         """
218         img = Image.open(path).convert("L")
219         img = img.resize((self.img_size, self.img_size), Image.LANCZOS)
220
221         arr = np.array(img, dtype=np.float64)
222
223         # Percentile-based contrast stretching (within the circular mask)
224         mask_pixels = arr[self.mask > 0.5]
225         lo, hi = np.percentile(mask_pixels, 2), np.percentile(mask_pixels,
226             98)
227         if hi - lo > 1:
228             arr = np.clip((arr - lo) / (hi - lo), 0.0, 1.0) * 255.0
229
230         return arr
231
232     # ---- Importance / edge map
233     -----
234
235     def _compute_importance_map(self) -> np.ndarray:
236         """Edge-aware importance weighting via Sobel magnitude.

```

```

235
236     Pixels near edges get higher weight so the algorithm prioritises
237     reproducing structural details over flat shading.
238     """
239     if self.importance_edge_weight <= 0:
240         return self.mask.copy()

```

- **Line 214–221:** 알고리즘이 단순한 배경 명암보다 구조적인 형태와 세부 디테일 (Edge)에 끈을 더 많이 할당하도록 가중치 지도를 생성합니다. 사용자가 가중치를 0으로 주었다면 마스크와 동일한 균일 맵을 반환합니다.
- **Line 223–232:** PIL의 ImageFilter.Kernel을 직접 정의하여 수평 (sx), 수직 (sy) 방향 소벨 (Sobel) 그라디언트를 검출하고 에지의 크기 (magnitude) 벡터 맵을 계산합니다. 필터 에러 방지를 위해 기본 제공되는 FIND_EDGES를 예비 처리용으로 둡니다.
- **Line 234–241:** 계산된 에지 맵을 최댓값으로 나누어 점수를 0~1로 정규화한 뒤, 사용자의 강조 인자 (importance_edge_weight)를 곱합니다. 결과적으로 평탄한 곳에는 가중치 1.0, 아주 뚜렷한 윤곽이 있는 곳엔 $1.0 + \text{weight}$ 형태의 가중치가 부여되게 됩니다.

블록 7: 기하학적 보조 함수들 (마스크 및 핀 배치) (Lines 242–261)

```

242     img_u8 = np.clip(self.target, 0, 255).astype(np.uint8)
243     pil_img = Image.fromarray(img_u8, mode="L")
244
245     try:
246         sx = pil_img.filter(ImageFilter.Kernel(
247             (3, 3), [-1, 0, 1, -2, 0, 2, -1, 0, 1], scale=1, offset
248             =128
249         ))
250         sy = pil_img.filter(ImageFilter.Kernel(
251             (3, 3), [-1, -2, -1, 0, 0, 0, 1, 2, 1], scale=1, offset
252             =128
253         ))
254         ex = np.array(sx, dtype=np.float64) - 128.0
255         ey = np.array(sy, dtype=np.float64) - 128.0
256         edge_mag = np.sqrt(ex ** 2 + ey ** 2)
257     except Exception:
258         edges = pil_img.filter(ImageFilter.FIND_EDGES)
259         edge_mag = np.array(edges, dtype=np.float64)
260
261     emax = edge_mag.max()
262     if emax > 0:
263         edge_mag /= emax

```

- **Line 245–250:** 정방형 이미지 내에서 핀들이 박혀 이뤄지는 원 영역을 가리기 위해, 중심점 (centre) 기준으로 원 안에 포함되는 픽셀엔 1, 바깥엔 0을 가지는 논리 배열 (mask)을 생성합니다. Numpy의 ogrid를 써서 고속으로 처리합니다.
- **Line 252–261:** 극좌표계의 원리를 사용해 원 둘레를 num_pins 개수만큼 균일한 각도 (np.linspace) 조각으로 분리하고, 삼각함수 (cos, sin)를 사용해 x, y 픽셀 좌표 공간으로 변환합니다. 이미지 배치를 벗어나지 않도록 np.clip으로 제한하고 결과 좌표를 배열에 담아 저장합니다.

블록 8: 사전 계산 (선분 픽셀 캐시 및 이웃 핀 목록) (Lines 262–298)

```

262         importance = 1.0 + self.importance_edge_weight * edge_mag
263         importance *= self.mask
264         return importance
265
266     # ---- Geometry helpers
267     -----
268
269     def _circular_mask(self) -> np.ndarray:
270         centre = self.img_size / 2.0
271         radius = centre - 1
272         Y, X = np.ogrid[: self.img_size, : self.img_size]
273         return ((X - centre) ** 2 + (Y - centre) ** 2 <= radius ** 2).
274             astype(
275                 np.float64
276             )
277
278     def _compute_pin_positions(self) -> list[tuple[int, int]]:
279         """Return (row, col) for each pin, evenly spaced on a circle."""
280         centre = self.img_size / 2.0
281         radius = centre - 1
282         angles = np.linspace(0, 2 * np.pi, self.num_pins, endpoint=False)
283         pins = []
284         for a in angles:
285             c = int(np.clip(centre + radius * np.cos(a), 0, self.img_size
286                             - 1))
287             r = int(np.clip(centre + radius * np.sin(a), 0, self.img_size
288                             - 1))
289             pins.append((r, c))
290         return pins
291
292     # ---- Line pre-computation
293     -----
294
295     def _precompute_lines(self):
296         """Cache anti-aliased pixel coords for every valid pin pair (
297             parallel)."""
298         print(f"Pre-computing line pixel coordinates ({self._num_workers}
299             workers) ...")
300         tasks = []
301         for i in range(self.num_pins):
302             for j in range(i + 1, self.num_pins):
303                 arc = min(abs(i - j), self.num_pins - abs(i - j))
304                 if arc < self.min_pin_distance:

```

- **Line 264–284:** 가장 빈번하게 호출되는 후보 평가 구간의 부하를 줄이기 위한 핵심 병렬 로직입니다. 등글게 배치된 핀 인덱스 (i, j) 에 대해 사이 거리(arc)를 파악하고 `min_pin_distance` 이상 떨어진 모든 유효 쌍의 픽셀 좌표 래스터화 태스크를 리스트 업합니다. 이를 `multiprocessing.Pool`에 던져 다중 코어로 한 번에 처리한 후, 거대한 `self.line_cache` 딕셔너리에 핀 튜플 키 단위로 저장합니다. 10만 개가 넘을 수 있는 쌍을 단수 초 만에 파싱합니다.
- **Line 286–298:** 위 선분 캐싱 조건에서 합격한 쌍들을 바탕으로, 각 핀 i 에 대해 "해당 위치에서 나갈 수 있는 후보 목적지 핀"을 필터링해 둔 `pin_neighbours` 이중 리스트를 만듭니다. 탐색 매 단계에서 후보를 전체 풀 탐색하는 대신 단숨에 쳐낼 수 있도록 합니다.

블록 9: L2 에너지 및 최적화 점수 (Error Reduction) 계산 (Lines 299–343)

```

300         continue
301         r0, c0 = self.pin_coords[i]
302         r1, c1 = self.pin_coords[j]
303         tasks.append((i, j, r0, c0, r1, c1, self.img_size))
304
305     with multiprocessing.Pool(processes=self._num_workers) as pool:
306         results = pool.map(_compute_line_worker, tasks, chunksize=256)
307
308     for key, line_data in results:
309         self.line_cache[key] = line_data
310
311     print(f" → {len(self.line_cache):,} valid lines cached.")
312
313 def _precompute_neighbours(self) -> list[list[int]]:
314     """For every pin, list valid destination pins (respecting min
315     distance)."""
316     neighbours: list[list[int]] = [[] for _ in range(self.num_pins)]
317     for i in range(self.num_pins):
318         for j in range(self.num_pins):
319             if i == j:
320                 continue
321             arc = min(abs(i - j), self.num_pins - abs(i - j))
322             if arc >= self.min_pin_distance:
323                 key = (min(i, j), max(i, j))
324                 if key in self.line_cache:
325                     neighbours[i].append(j)
326
327     return neighbours
328
329 # ---- Energy & scoring
330 -----
331
332 def _mse(self, transmittance: np.ndarray) -> float:
333     """Mean squared error (unweighted, within circular mask)."""
334     rendered = 255.0 * transmittance
335     diff = (rendered - self.target) * self.mask
336     n_pixels = np.sum(self.mask)
337     return float(np.sum(diff ** 2) / (n_pixels + 1e-10))
338
339 def _weighted_energy(self, transmittance: np.ndarray) -> float:
340     """Importance-weighted sum of squared errors."""
341     rendered = 255.0 * transmittance
342     diff = rendered - self.target
343     return float(np.sum(self.importance * diff ** 2))
344
345 def _line_error_reduction(
346     self, transmittance: np.ndarray, pin_a: int, pin_b: int
347 ) -> float:

```

- **Line 301–306:** 현재 투과율(transmittance) 캔버스가 완벽히 복원됐을 때 목표(target) 대비 화소의 MSE (차이 제곱의 평균, unweighted)를 계산합니다. 로그와 진행 경과 출력에 사용됩니다.
- **Line 308–312:** 위에서 미리 구한 중요도 에지 맵을 곱하여 시각적으로 민감한 부분의 오차가 더 높은 에너지 페널티를 받도록 만든 지표 함수입니다.

- **Line 314–343:** 한 가닥의 선($a \rightarrow b$)을 그을 때 발생하는 전체 가중치 에러 감소량(**Error Reduction**)을 유도된 폐형식(closed-form) 식으로 단 한 번에 계산하는 가장 핵심적인 수학 연산 최적화 함수입니다. 선이 지나가는 픽셀 위치의 밝기, 가중치를 가져온 뒤 벡터 배열 연산($W * s * (2.0 * d - s)$)으로 합산하여 양수가 반환되면 에러가 줄어듦(이득), 음수이면 오히려 화질이 훼손됨을 뜻합니다.

블록 10: 후보 핀 평가 함수 (병렬 스레드 지원) (Lines 344–373)

```

344     """Compute the reduction in weighted L2 error from adding a string
345     .
346     Closed-form derivation (multiplicative opacity model):
347         R_i = 255 · T_i          (current rendered value at pixel i)
348         R'_i = R_i · (1 - ·v_i) (value after adding the string)Δ
349         E = Σ W_i · s_i · (2·d_i - s_i)
350     where
351         d_i = R_i - target_i    (positive when pixel is too bright)
352         s_i = R_i · ·v_i       (amount of darkening applied)
353
354     Returns positive value when the string reduces the error.
355     """
356     key = (min(pin_a, pin_b), max(pin_a, pin_b))
357     if key not in self.line_cache:
358         return -np.inf
359     rr, cc, val = self.line_cache[key]
360     if len(rr) == 0:
361         return 0.0
362
363     R = 255.0 * transmittance[rr, cc] # current rendered
364     T = self.target[rr, cc]           # target
365     W = self.importance[rr, cc]       # importance weights
366     a = self.string_opacity * val     # effective per-pixel opacity
367
368     s = R * a                         # darkening amount
369     d = R - T                         # residual (>0 means too
370         bright)
371
372     # ΔE = Σ W·s·(2d - s) - positive ↔ error decreases
373     return float(np.sum(W * s * (2.0 * d - s)))

```

- **Line 346–358:** 실 짜기를 시작할 가장 완벽한 0번 핀을 찾기 위한 자동 설정 함수입니다. 스레드풀(ThreadPoolExecutor)을 이용, 0번부터 N번까지의 출발선에서 최선의 감소량을 가진 핀을 스캔하여 반환합니다.
- **Line 360–373:** 단일 핀 current_pin에서 나갈 수 있는 타겟 candidates 집단에 대해 스코어를 테스트하는 실무 평가 함수입니다. 단순히 감소량(score)만을 보는 게 아니라, 이미 쓴 핀 쌍 정보(pair_usage)를 조회한 뒤 반복 재사용 횟수마다 강한 배수형 감쇠 페널티($1.0 - 0.2 * usage$)를 맥여 동일 구역이 새까맣게 타는 현상(과암화)을 억제합니다.

블록 11: 주 알고리즘(Main: Phase 1) — 탐욕 선택 로직 (Lines 374–455)

```

374     # ---- Starting-pin selection
375     -----

```

```

376 def _find_best_start(self, transmittance: np.ndarray) -> int:
377     """Pick the starting pin whose best outgoing line offers the most
378     error reduction (parallelised across CPU cores)."""
379     def _eval_pin(pin):
380         best = -np.inf
381         for cand in self.pin_neighbours[pin]:
382             sc = self._line_error_reduction(transmittance, pin, cand)
383             if sc > best:
384                 best = sc
385         return pin, best
386
387     with ThreadPoolExecutor(max_workers=self._num_workers) as pool:
388         results = list(pool.map(_eval_pin, range(self.num_pins)))
389
390     return max(results, key=lambda x: x[1])[0]
391
392 def _eval_candidates(self, transmittance, current_pin, candidates,
393 pair_usage):
394     """Evaluate a chunk of candidate pins (thread worker)."""
395     best_score = -np.inf
396     best_pin = -1
397     for cand in candidates:
398         score = self._line_error_reduction(transmittance, current_pin,
399 cand)
400         pair = (min(current_pin, cand), max(current_pin, cand))
401         usage = pair_usage.get(pair, 0)
402         if usage > 0:
403             score *= max(0.05, 1.0 - 0.2 * usage)
404         if score > best_score:
405             best_score = score
406             best_pin = cand
407     return best_pin, best_score
408
409 # ---- Main algorithm
410 -----
411
412 def run(self, verbose: bool = True):
413     """Execute the energy-minimising string art algorithm.
414
415     Phase 1 - Greedy construction:
416         Iteratively add the string that maximises weighted L2 error
417         reduction, using a multiplicative opacity model.
418
419     Phase 2 - Refinement:
420         Replay the sequence from scratch, re-evaluating every string
421         choice in its updated context and swapping in better
422         alternatives where possible.
423
424     Returns
425     -----
426     string_sequence : list[int]
427         Ordered pin indices.
428     result_image : np.ndarray
429         Final rendered image (0-255, normal orientation).
430     """
431     # Transmittance starts at 1.0 everywhere (fully white canvas)
432     transmittance = np.ones(
433         (self.img_size, self.img_size), dtype=np.float64

```

```

431     )
432     pair_usage: dict[tuple[int, int], int] = {}
433
434     self._thread_pool = ThreadPoolExecutor(max_workers=self.
435         _num_workers)
436
437     if verbose:
438         print()
439         print("=" * 60)
440         print("  STRING ART - Energy Minimisation (Birsak et al. 2018)"
441             )
442         print("=" * 60)
443         print(f"  Pins                : {self.num_pins}")
444         print(f"  Max strings         : {self.max_strings}")
445         print(f"  String opacity      : {self.string_opacity:.3f}")
446         print(f"  Min pin distance    : {self.min_pin_distance}")
447         print(f"  Image size          : {self.img_size}x{self.img_size}"
448             )
449         print(f"  Edge importance     : {self.importance_edge_weight:.1f
450             }")
451         print(f"  Refinement rounds   : {self.refinement_rounds}")
452         print(f"  CPU workers         : {self._num_workers}")
453         init_mse = self._mse(transmittance)
454         print(f"  Initial MSE         : {init_mse:.1f}")
455         print()
456
457     # =====
458     # Phase 1: Greedy construction
459     # =====

```

- **Line 376–408:** run 메인 드라이버의 서두입니다. 캔버스의 투과율 변수를 모든 픽셀에서 1.0(백색)으로 초기화한 후 시작 핀을 결정해 시퀀스 리스트(string_sequence)에 넣고 터미널에 초기 정보를 인쇄합니다.
- **Line 411–427:** 지정된 max_strings 값만큼 루프를 도는 **Phase 1 탐욕(Greedy)** 알고리즘 구간입니다. 코어 수(_num_workers)를 계산해 후보 리스트(candidates)를 chunk_sz 단위로 쪼개 각 스레드에 던져 병렬 평가합니다. 파이썬 GIL 한계에서도 I/O 대기가 아닌 Numpy C모듈 기반 계산을 사용해 어느정도 스레드풀 이점을 취합니다.
- **Line 429–442:** 평가 결과 최고의 점수가 0 이하라면 실을 그어도 손해만 난다는 의미로, 지능적 즉시 중단(break) 기준을 발동시킵니다. 합격했다면 찾은 핀 섹션의 픽셀 좌표마다 투과율을 비율로 감소시킵니다. 가산적 뺄셈(-)이 아닌 투과율 복사 법칙에 의한 곱셈(*(1.0 - opacity*val))임이 이 알고리즘의 정체성입니다.
- **Line 444–455:** 사용한 라인 쌍을 카운팅하고 핀을 이동합니다. 점수를 score_history에 기록해 최근 800번의 점수 평균이 극초기 평균 대비 0.5% 아래로 80회 이상 떨어졌다면 이미 완전한 채도 포화(정체)에 빠졌다고 통계적으로 확신하여 조기 종료(Early stopping)를 시킵니다.

블록 12: 정련(Phase 2 Refinement) 단계 및 실행 마무리 (Lines 456–542)

```

456     if verbose:
457         print("  ——— Phase 1: Greedy Construction ———")
458
459     current_pin = self._find_best_start(transmittance)

```

```

460 self.string_sequence = [current_pin]
461
462 if verbose:
463     print(f" Best starting pin : {current_pin}")
464
465 score_history: list[float] = []
466 stall_count = 0
467
468 for it in range(self.max_strings):
469     # Evaluate candidates in parallel across CPU cores
470     candidates = self.pin_neighbours[current_pin]
471     n_w = min(self._num_workers, max(1, len(candidates) // 8))
472     if n_w <= 1 or len(candidates) < 16:
473         best_pin, best_score = self._eval_candidates(
474             transmittance, current_pin, candidates, pair_usage
475         )
476     else:
477         chunk_sz = max(1, len(candidates) // n_w)
478         chunks = [candidates[i:i + chunk_sz]
479                     for i in range(0, len(candidates), chunk_sz)]
480         futures = [self._thread_pool.submit(
481             self._eval_candidates, transmittance, current_pin,
482             chunk, pair_usage) for chunk in chunks]
483         results = [f.result() for f in futures]
484         best_pin, best_score = max(results, key=lambda x: x[1])
485
486     # ——— Stopping: no error-reducing string found ———
487     if best_score <= 0 or best_pin == -1:
488         if verbose:
489             print(
490                 f" STOP at string {it}: "
491                 f"no error-reducing string found"
492             )
493             break
494
495     # ——— Apply string (multiplicative opacity) ———
496     key = (min(current_pin, best_pin), max(current_pin, best_pin))
497     rr, cc, val = self.line_cache[key]
498     transmittance[rr, cc] *= (1.0 - self.string_opacity * val)
499
500     pair_usage[key] = pair_usage.get(key, 0) + 1
501     self.string_sequence.append(best_pin)
502     current_pin = best_pin
503
504     # ——— Adaptive stopping based on score trend ———
505     score_history.append(best_score)
506     window = 800
507     if len(score_history) > window:
508         recent_avg = np.mean(score_history[-window:])
509         early_avg = np.mean(score_history[:window])
510         if recent_avg < 0.005 * early_avg:
511             stall_count += 1
512             if stall_count > 80:
513                 if verbose:
514                     print(
515                         f" STOP at string {it}: "
516                         f"negligible improvement"
517                     )

```

```

518         break
519     else:
520         stall_count = 0
521
522     # Progress reporting
523     if verbose and (it + 1) % 1000 == 0:
524         cur_mse = self._mse(transmittance)
525         print(
526             f"      {it + 1:>5}/{self.max_strings} |      "
527             f"MSE {cur_mse:7.1f} |      "
528             f"Δ {best_score:10.1f} |      "
529             f"pairs {len(pair_usage):,}"
530         )
531
532     n_greedy = len(self.string_sequence) - 1
533     if verbose:
534         greedy_mse = self._mse(transmittance)
535         print(
536             f" Greedy done: {n_greedy} strings, MSE = {greedy_mse:.1f}
537             )"
538         print()
539
540     # =====
541     # Phase 2: Refinement (re-evaluate each string in updated context
542     # =====

```

- **Line 458–499:** 전체 문자열이 놓인 이후 초기에 그랬던 실들에는 편향이 존재할 가능성이 높습니다. 사용자가 설정한 `refinement_rounds` 횟수만큼 반복하며 새 하얀 캔버스를 임시로 두고 초기 순서대로 핀을 지나는 시뮬레이션을 재현합니다. 단, 다음 핀으로 움직일 때 기존 선택지(`orig_dest`) 점수와 변경된 현재 문맥에서의 완전한 새로운 최적 목적지(`cand_pin`) 대안을 비교 평가합니다.
- **Line 501–517:** 새로운 대처안의 L2 감소 점수가 원본보다 크다면 핀 선로를 교체(Swap) 함으로써 후처리 최적화를 단행합니다. 대안이건 기존값이건 모두 마이너스 점수라면 스킵하지만, 물리적으로 실의 연결 통로는 유지해야 하기 때문에 투과율은 안 내리고 위치만 유지합니다.
- **Line 519–542:** 알고리즘이 결정한 총 누적 투과율을 255.0에 곱하여 눈에 띄는 정수 흑백 이미지(`result_image`)로 생성 및 복원합니다. 객관적 화질 평가 지표인 종료 MSE와 PSNR (dB 데시벨)을 계산하여 콘솔에 띄운 후 스레드풀 서버를 닫으며 알고리즘 동작을 완전히 매듭짓습니다.

블록 13: 렌더링 시각화 모듈 (Matplotlib PIL) (Lines 543–580)

```

543     for ref_round in range(self.refinement_rounds):
544         if verbose:
545             print(
546                 f" ——— Phase 2: Refinement "
547                 f"(round {ref_round + 1}/{self.refinement_rounds})
548                 ———"
549             )
550
551     trans_new = np.ones(

```

```

551         (self.img_size, self.img_size), dtype=np.float64
552     )
553     new_seq = [self.string_sequence[0]]
554     pair_usage_new: dict[tuple[int, int], int] = {}
555     improved = 0
556     skipped = 0
557
558     n_total = len(self.string_sequence) - 1
559     for i in range(n_total):
560         src_pin = new_seq[-1]
561
562         # Score the original destination
563         orig_dest = self.string_sequence[i + 1]
564         orig_score = self._line_error_reduction(
565             trans_new, src_pin, orig_dest
566         )
567         orig_key = (min(src_pin, orig_dest), max(src_pin,
568             orig_dest))
569
570         # Score the best alternative from this position (parallel)
571         candidates = self.pin_neighbours[src_pin]
572         n_w = min(self._num_workers, max(1, len(candidates) // 8))
573         if n_w <= 1 or len(candidates) < 16:
574             cand_pin, cand_score = self._eval_candidates(
575                 trans_new, src_pin, candidates, pair_usage_new
576             )
577         else:
578             chunk_sz = max(1, len(candidates) // n_w)
579             chunks = [candidates[i:i + chunk_sz]
580                 for i in range(0, len(candidates), chunk_sz)
581             ]
582             futures = [self._thread_pool.submit(

```

- **Line 545–553:** `render_pil` 파트입니다. 투과율 곱 알고리즘이 계산용 가상 이미지였다면, 이 메서드는 도출된 수천 개의 핀 순서(`string_sequence`) 정보를 토대로 Python `ImageDraw`를 이용해 가장 선명성이 높은 정통 벡터형 (Aliasing free) 직선 아트로 그림을 복원하고 평가용 매트릭스로 추출합니다.
- **Line 555–580:** `show_results` 함수가 세 개의 서브플롯 (Original, Algorithm Simulated, Vector Render) 이미지를 나란히 배치해 시각적으로 비교해 주는 `matplotlib` 플롯 (Dashboard)을 렌더링하고 사용자가 원할 경우 PNG에 자동 저장시킵니다.

블록 14: 작업 파일 내보내기 (TXT 및 좌표 Export) (Lines 581–680)

```

581         self._eval_candidates, trans_new, src_pin,
582         chunk, pair_usage_new) for chunk in chunks]
583     results = [f.result() for f in futures]
584     cand_pin, cand_score = max(results, key=lambda x: x
585         [1])
586
587     if cand_score > orig_score:
588         best_score = cand_score
589         best_pin = cand_pin
590     else:
591         best_score = orig_score
592         best_pin = orig_dest

```



```

592
593     # Skip string entirely if even the best is unhelpful
594     if best_score <= 0:
595         # Still move to the original destination (thread
596             continuity)
597         new_seq.append(orig_dest)
598         skipped += 1
599         continue
600
601     if best_pin != orig_dest:
602         improved += 1
603
604     # Apply the chosen string
605     key = (min(src_pin, best_pin), max(src_pin, best_pin))
606     rr, cc, val = self.line_cache[key]
607     trans_new[rr, cc] *= (1.0 - self.string_opacity * val)
608     pair_usage_new[key] = pair_usage_new.get(key, 0) + 1
609     new_seq.append(best_pin)
610
611     self.string_sequence = new_seq
612     transmittance = trans_new
613
614     if verbose:
615         ref_mse = self._mse(transmittance)
616         print(
617             f"        Swapped {improved} strings, "
618             f"skipped {skipped}, MSE = {ref_mse:.1f}"
619         )
620         print()
621
622     # =====
623     # Final result
624     # =====
625     self.result_image = np.clip(255.0 * transmittance, 0, 255)
626     self.transmittance = transmittance
627
628     n_strings = len(self.string_sequence) - 1
629     if verbose:
630         final_mse = self._mse(transmittance)
631         psnr = 10.0 * np.log10(255.0 ** 2 / (final_mse + 1e-10))
632         print(f"        ")
633         print(f"    Final : {n_strings} strings")
634         print(f"    MSE   : {final_mse:.1f}")
635         print(f"    PSNR  : {psnr:.1f} dB")
636         print(f"        ")
637         print()
638
639     self._thread_pool.shutdown(wait=False)
640
641     return self.string_sequence, self.result_image
642
643     # ---- Visualisation
644     -----
645
646     def render_pil(self, line_thickness: int = 1) -> np.ndarray:
647         """Re-render the result with PIL (crisp vector-style lines)."""
648         img = Image.new("L", (self.img_size, self.img_size), 255)
649         draw = ImageDraw.Draw(img)

```

```

648         for i in range(len(self.string_sequence) - 1):
649             pa, pb = self.string_sequence[i], self.string_sequence[i + 1]
650             r0, c0 = self.pin_coords[pa]
651             r1, c1 = self.pin_coords[pb]
652             draw.line([(c0, r0), (c1, r1)], fill=0, width=line_thickness)
653         return np.array(img)
654
655     def show_results(self, save_path: str | None = None):
656         """Side-by-side comparison: target → algorithm output → PIL
657            render."""
658         fig, axes = plt.subplots(1, 3, figsize=(18, 6))
659
660         axes[0].imshow(self.target, cmap="gray", vmin=0, vmax=255)
661         axes[0].set_title("Target image")
662         axes[0].axis("off")
663
664         if self.result_image is not None:
665             n = len(self.string_sequence) - 1
666             mse = self._mse(self.transmittance) if self.transmittance is
667                 not None else 0
668             axes[1].imshow(self.result_image, cmap="gray", vmin=0, vmax
669                 =255)
670             axes[1].set_title(f"String art ({n} strings, MSE={mse:.1f})")
671             axes[1].axis("off")
672
673         rendered = self.render_pil()
674         axes[2].imshow(rendered, cmap="gray", vmin=0, vmax=255)
675         axes[2].set_title("PIL vector render")
676         axes[2].axis("off")
677
678         plt.tight_layout()
679         if save_path:
680             plt.savefig(save_path, dpi=150, bbox_inches="tight")
681             print(f" Figure saved → {save_path}")
682         plt.show()

```

- **Line 583–592:** 생성된 작품을 실제 못-실 조립 자동화 기계나 손으로 직접 만들기 위해서, 노드 인덱스가 순서대로 적힌 텍스트 지시서(pin_sequence.txt)를 출력합니다.
- **Line 594–634:** 로봇 팔 작동이나 AutoCAD CAM 도면화를 위해 좌표계 내보내기(save_coordinates)를 수행합니다. 좌하단 구석이 아닌 물리 정중앙 중심 좌표계(0, 0)와 실제 밀리미터(mm) 단위를 적용, 직교 벡터 좌표 위치(x, y)로 핀 번호와 시작/도착 세트 위치를 매핑해 문서로 떨어줍니다.
- **Line 636–680:** CNC 가공이나 수동 제작용 작업 베이스 판으로 쓰일 스케일업(2000×2000 픽셀 사이즈) 템플릿(pin_template.png)을 포맷 렌더링합니다. 윤곽 외각선, 각 핀의 스폿서클뿐만 아니라 번호 겹침을 방지하고자 규칙적인 주기(label_every)별로 인덱스 아라비아 숫자를 투사해 넣습니다.

블록 15: CLI 메인 동작부 (Entry Point) (Lines 681–745)

```

681     # ---- Export
682     -----
683     def save_sequence(self, filepath: str):
684         """Write the pin sequence to a plain-text file."""

```

```

685     with open(filepath, "w", encoding="utf-8") as f:
686         f.write(f"# Computational String Art - Pin Sequence\n")
687         f.write(f"# Pins:      {self.num_pins}\n")
688         f.write(f"# Strings: {len(self.string_sequence) - 1}\n")
689         f.write(f"# Opacity: {self.string_opacity}\n")
690         f.write(f"# Format:  one pin index per line (0-indexed)\n\n")
691         for pin in self.string_sequence:
692             f.write(f"{pin}\n")
693     print(f" Pin sequence saved → {filepath}")
694
695 def save_coordinates(self, filepath: str, frame_diameter_mm: float =
696 600.0):
697     """Write the string sequence as physical (x, y) coordinates in mm.
698
699     The circle is centred at (0, 0) with the given diameter.
700     Each line in the file represents one string connection:
701         string_number, from_pin, x_from, y_from, to_pin, x_to, y_to
702
703     A header section lists all pin positions first.
704     """
705     radius_mm = frame_diameter_mm / 2.0
706     centre = self.img_size / 2.0
707     img_radius = centre - 1
708
709     # Convert pixel pin coords → mm coords (origin at centre)
710     pin_mm = []
711     for r, c in self.pin_coords:
712         x_mm = (c - centre) / img_radius * radius_mm
713         y_mm = -(r - centre) / img_radius * radius_mm # y-up
714         pin_mm.append((x_mm, y_mm))
715
716     with open(filepath, "w", encoding="utf-8") as f:
717         f.write(f"# String Art - Pin Coordinates (mm)\n")
718         f.write(f"# Frame diameter : {frame_diameter_mm:.1f} mm\n")
719         f.write(f"# Pins           : {self.num_pins}\n")
720         f.write(f"# Strings          : {len(self.string_sequence) - 1}\n")
721         f.write(f"# Origin           : centre of circle (0, 0)\n")
722         f.write(f"# Y-axis          : positive = up\n")
723         f.write(f"#\n")
724         f.write(f"# — PIN POSITIONS —\n")
725         f.write(f"# pin_index, x_mm, y_mm\n")
726         for idx, (x, y) in enumerate(pin_mm):
727             f.write(f"{idx}, {x:.2f}, {y:.2f}\n")
728         f.write(f"#\n")
729         f.write(f"# — STRING SEQUENCE (coordinates) —\n")
730         f.write(f"# string_no, from_pin, x_from, y_from, to_pin, x_to, y_to\n")
731         for i in range(len(self.string_sequence) - 1):
732             pa = self.string_sequence[i]
733             pb = self.string_sequence[i + 1]
734             x0, y0 = pin_mm[pa]
735             x1, y1 = pin_mm[pb]
736             f.write(f"{i + 1}, {pa}, {x0:.2f}, {y0:.2f}, {pb}, {x1:.2f}, {y1:.2f}\n")
737
738     print(f" Coordinates saved → {filepath}")

```

```

738         print(f"      Frame: {frame_diameter_mm:.0f} mm diameter, {self.
739               num_pins} pins")
740
741     def save_pin_template(self, filepath: str, canvas_px: int = 2000):
742         """Generate a high-res pin-coordinate template image for
743            fabrication.
744
745         Draws:
746         - A circle showing the frame boundary.
747         - Numbered dots at each pin position.

```

- **Line 683–691:** 사용자가 `python 1st review.py` 를 바로 칠 때 작동하는 공간으로 메인 기본 파라미터 변수들을 하드코딩해 세팅해 주고 있습니다.
- **Line 693–703:** 커맨드 라인 실행인자(`sys.argv`)가 있다면 입력 파라미터를 파싱해 사용하며, 만약 없다면 스크립트 실행 경로를 획득한 후 기본적으로 하드 코딩된 샘플 원본 사진 이미지 파일을 찾아 대상으로 연결시킵니다.
- **Line 705–745:** 현재 세팅된 파라미터를 예쁘게 꾸며진 CLI 배너로 띄워 주며 `StringArt` 클래스 인스턴스(`sa`)를 만들어 주입합니다. 그런 다음 `sa.run()`을 호스트하고 이어서 위에서 정의했던 `sequence`, `coordinates`, `template` 3종 세트와 화상 결과 파일을 파일 시스템에 써서 사용자에게 결과물 패키지를 제공해 줍니다.

References

- [1] Michael Birsak et al., *String Art: Towards Computational Fabrication of String Images*, Eurographics, 2018.
- [2] Xiaolin Wu, *An Efficient Antialiasing Technique*, SIGGRAPH Proceedings, 1991.